Kafka

什麼是 Kafka

你很可能聽過 Kafka，它很紅。根據官網，財星 100 大企業裡有 80% 在用。既然能撐起世界上最大規模的公司，拿來應付你的下一場 system design 面試也夠用了。

Apache Kafka 是開源、分散式的 event streaming 平台，可以當成 message queue 用，也可以當成 stream processing 系統。Kafka 的強項是高效能、可擴展和持久化；在設定得當（例如 replication 和 acknowledgment）的情況下，可以對訊息遺失提供很強的保證。

這份講義會用「由上而下」的方式講：先看 Kafka 的全貌，再一層一層進細節。如果你已經會基礎，可以直接往後翻到進階段落。

情境範例

假設現在是世足賽（我個人最愛的比賽）。我們經營一個提供比賽即時數據的網站，每次進球、球員吃牌或換人，我們都想立刻更新網站。

事件發生時會被放進一個 queue。負責把這些事件放進 queue 的伺服器或程式，我們叫它 **producer**。下游有一台伺服器從 queue 讀事件、更新網站，我們叫它 **consumer**。

接著想像世足從 48 隊擴大成假想的 1000 隊，而且所有比賽同時開打。事件量暴增，單一負責 queue 的伺服器撐不住，consumer 也像被消防水柱灌一樣被壓垮。

我們得加更多伺服器來分散 queue、做水平擴展，但要怎麼保證事件還是「照順序」被處理？

buildmoat.org

如果隨機把事件分到各台伺服器，會一團亂：可能比賽還沒開始就出現進球，或球員還沒犯規就被記牌。

比較合理的做法是：依照「屬於哪一場比賽」來分配 queue 裡的項目。這樣同一場比賽的所有事件都在同一條 queue 上，自然能按順序處理。這也是 Kafka 的核心概念之一：**透過 Kafka 收發的訊息，會用 partitioning strategy 分散到不同 partition**（Kafka 有預設策略，但選對 key 對順序保證很重要）。

那 consumer 還是負載過高怎麼辦？多加幾台 consumer 不難，但要怎麼確保每個事件只被處理一次？我們可以把多個 consumer 組成 Kafka 所說的 **consumer group**。在 consumer group 裡，每個 partition 只會分配給其中一個 consumer，所以正常情況下每個事件只會送給一個 consumer。（在故障情境下，Kafka 預設是 at-least-once，訊息可能被重複處理，但不會被拆給多個 consumer。）

最後，假設我們要把這個「世足」擴大成更多運動，例如籃球。我們不想讓足球網站顯示籃球事件，也不想讓籃球網站顯示足球事件，於是引入 **topic** 的概念：每個事件都掛在某個 topic 下，consumer 可以只訂閱特定的 topic。所以更新足球網站的 consumer 只訂閱足球 topic，更新籃球網站的只訂閱籃球事件。

buildmoat.org

基本術語與架構

上面的例子很好懂，接下來把 Kafka 的關鍵名詞講清楚一點。

一個 Kafka cluster 由多台 **broker** 組成，就是一台台獨立的伺服器（實體或虛擬都可以）。每台 broker 負責存資料和服務 client；broker 越多，能存的資料越多、能服務的 client 也越多。

每台 broker 上有多個 **partition**。每個 partition 是一串「只能追加」、不可變的訊息序列，可以想成 log 檔。Kafka 能 scale，靠的就是 partition：訊息可以並行地被多個 partition 消化。

Topic 則是 partition 的邏輯分組。在 Kafka 裡，你是透過 topic 來 publish 和 subscribe 資料：發訊息時發到某個 topic，消費時從某個 topic 讀。Topic 一定是 multi-producer，也就是一個 topic 可以有零個、一個或很多個 producer 寫入。

Topic 和 partition 有什麼差？Topic 是訊息的「邏輯」分組，partition 是「實體」分組。一個 topic 可以有多個 partition，每個 partition 可以在不同 broker 上。Topic 用來組織資料，partition 用來 scale 資料。

再來是 **producer** 和 **consumer**。Producer 負責把資料寫進 topic，consumer 負責從 topic 讀。Kafka 對兩者都提供簡單的 API，但訊息的「長什麼樣子、怎麼處理」是開發者的事，Kafka 只管存和送。

還有一點很重要：Kafka 可以當 message queue 用，也可以當 stream 用，兩者差別其實不大。兩種模式下 consumer 都用 offset commit 來記錄進度；差別在消費模式：當 message queue 用時，每則訊息由 group 裡的一個 consumer 處理後就算「被消費」；當 stream 用時，log 會保留、可以重播，多個 consumer group 可以各自獨立讀同一份資料，consumer 也可以持續即時處理新進資料。

Kafka 如何運作

當事件發生時，producer 會把訊息（也稱 record）組好、送到某個 Kafka topic。一則訊息有四個欄位，技術上都可以不填：value（payload）、key、timestamp、headers。Key 用來決定訊息要送到哪個 partition；timestamp 記錄建立或寫入時間（但 partition 內的順序是由 offset 決定，不是 timestamp）；headers 類似 HTTP headers，可以放 key-value 的 metadata。

buildmost.org

KAFKA Message	Record	Record Example
Headers	key-value metadata	source: "api" version: "2"
Key	determines partition	“BRA-ARG”
Value	payload	event: "goal" team: "ARG" player: "Messi" minute: 80
Timestamp	unix timestamp	1771820100000

雖然不是強制，但 key 會決定訊息進哪個 partition。如果你不提供 key，Kafka 會用預設策略把訊息分散到各 partition（現代 Kafka client 常用「sticky」partitioner：先集中到同一 partition 再輪替，以利批次效率）。所以在面試裡設計大型分散式系統時，你會想用 key 讓相關訊息進同一個 partition、保證順序。Key 的選擇很重要，後面還會提到。

下面是用 Kafka 指令列工具 `kafka-console-producer` 把訊息送到 topic `my-topic` 的範例：

```
kafka-console-producer --bootstrap-server localhost:9092 --
topic my_topic --property "parse.key=true" --property "key.
separator=:"
> key1: Hello, Kafka with key!
> key2: Another message with a different key
```

- `-property "parse.key=true"` 和 `-property "key.separator=:"` 用來指定 key 和 value 以冒號分隔。

用 Node.js 的 Kafka client `kafkajs` 寫起來像這樣：

```
// Initialize the Kafka client
const kafka = new Kafka({
  clientId: 'my-app',
  brokers: ['localhost:9092']
})
```

buildmoat.org

```
// Initialize the producer
const producer = kafka.producer()

const run = async () => {
  await producer.connect()
  await producer.send({
    topic: 'my_topic',
    messages: [
      { key: 'key1', value: 'Hello, Kafka with key!' },
      { key: 'key2', value: 'Another message with a differe
nt key' }
    ],
  })
}
```

當訊息被 publish 到 Kafka topic 時，Kafka 會先決定這則訊息要進哪個 partition。這個選擇很重要，因為它會影響資料在 cluster 裡的分佈，流程分兩步：

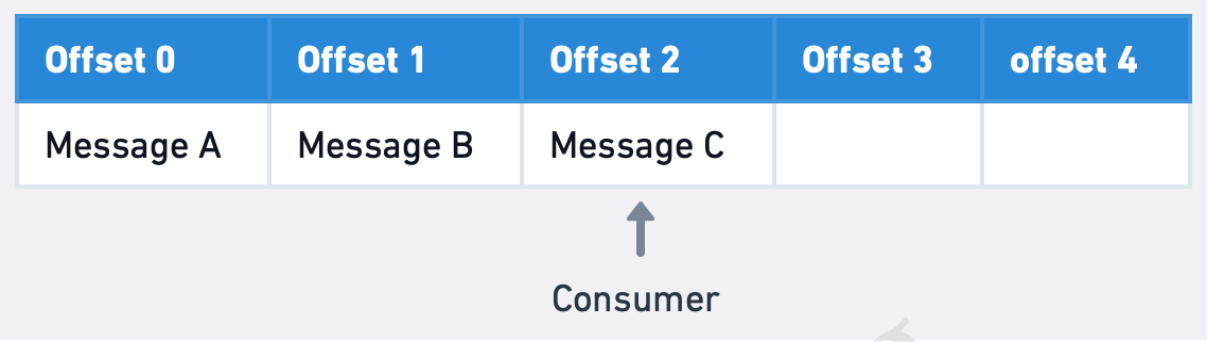
1. **Partition Determination（決定 partition）**：Kafka 用 partitioning 演算法對 message key 做 hash，把訊息分到某個 partition。如果沒有 key，可以 round-robin 到各 partition，或依 producer 設定的邏輯分配。同一 key 的訊息會進同一個 partition，在 partition 層級保證順序。
2. **Broker Assignment（對應到 broker）**：partition 決定後，Kafka 再根據 cluster metadata（由 cluster 裡的 controller 維護）找出該 partition 在哪台 broker 上，producer 會直接把訊息送給那台 broker。

Kafka 的每個 partition 本質上就是一支 append-only 的 log 檔，訊息依序寫到 log 尾端，所以 Kafka 常被形容成 distributed commit log。這種 append-only 設計帶來幾個好處：

1. **Immutability（不可變）**：寫入後 partition 裡的訊息不會被就地修改，只會依 retention 或 log compaction 被刪除。不可變對效能和可靠度很重要，也簡化 replication、加速 recovery，並避免「可改寫」系統常見的 consistency 問題。
2. **Efficiency（效能）**：只做尾端追加，可以減少磁碟 seek，而 seek 往往是儲存系統的瓶頸。
3. **Scalability（可擴展）**：append-only 的簡單模型有利水平擴展，可以加 partition、分散到多台 broker，也可以把每個 partition 複製到多台 broker 提升 fault tolerance。

buttondat.org

Partition 裡每則訊息都有一個唯一的 **offset**，代表它在該 partition 中的序號。Consumer 用 offset 記錄自己讀到哪裡，並定期把目前 offset commit 回 Kafka，這樣重啟或故障後可以從上次位置繼續。要注意：Kafka 預設是 at-least-once——如果 consumer 在處理完訊息後、commit offset 前掛掉，重啟後該訊息會被再處理一次。要達到 exactly-once 需要額外設定（idempotent producer + transactional API）。



訊息寫入對應的 partition 後，Kafka 透過 replication 機制保證持久與可用，採用 leader-follower 模型：

1. **Leader Replica Assignment**：每個 partition 有一個 leader replica，落在某台 broker 上，負責處理該 partition 的寫入（以及預設的讀取；Kafka 2.4+ 可以讓 consumer 從 follower 讀以優化延遲）。Leader 的分配由 cluster controller 集中管理，讓各 partition 的 leader 有效分散在 cluster 裡以平衡負載。
2. **Follower Replication**：除了 leader 外還有若干 follower replica，分布在不同 broker 上。Follower 不直接服務 client，而是被動從 leader 複製資料，作為備援，在 leader 掛掉時可以接手。
3. **Synchronization and Consistency**：Follower 持續與 leader 同步，以保有 partition log 的最新內容。Leader 故障時，已同步完成的某個 follower 可以被快速提升為新 leader，減少停機與資料損失。
4. **Controller's Role in Replication**：Cluster 內的 controller 負責監控所有 broker、管理 leader 與 replication。當某台 broker 故障，controller 會把 leader 角色交給某個 in-sync 的 follower，以維持 partition 可用。

Consumer 從 Kafka topic 讀訊息時用的是 **pull-based 模型**。不像有些系統會把資料 push 給 consumer，Kafka 的 consumer 是主動向 broker 拉新訊息，間隔由自己控制。Kafka 官方文件說明這是刻意的設計，好處包括：consumer 可以控制消費速率、簡化錯誤處理、避免拖慢較慢的 consumer、並利於批次處理。

延續前面的例子，下面是用 Kafka 指令列工具 `kafka-console-consumer` 從 `my-topic` 消費的寫法：

buildnoot.org

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic my_topic --from-beginning --property print.key=true --property key.separator=": "

# Output
key1: Hello, Kafka with key!
key2: Another message with a different key
```

用 `kafkajs` 從 `my_topic` 消費的寫法如下：

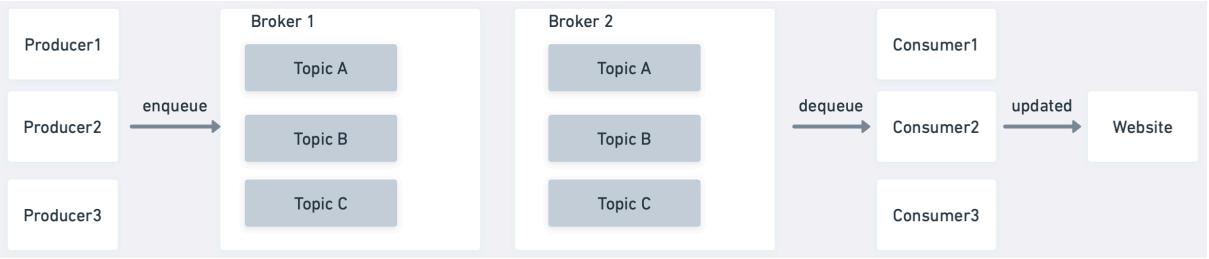
```
// Initialize the Kafka client
const kafka = new Kafka({
  clientId: 'my-app',
  brokers: ['localhost:9092']
})

// Initialize the consumer
const consumer = kafka.consumer({ groupId: 'my-group' })

const run = async () => {
  await consumer.connect()
  await consumer.subscribe({ topic: 'my_topic' })
  await consumer.run({
    eachMessage: async ({ topic, partition, message }) => {
      console.log({
        value: message.value?.toString(),
        key: message.key?.toString()
      })
    },
  })
}
```

把上述流程串起來，整體就是：producer 送訊息到 topic → 依 key 決定 partition → 寫入對應 broker 的 partition → replication 到 follower → consumer 用 pull 從 partition 讀、並 commit offset。

buildmoat.org



面試中何時該用 Kafka

Kafka 可以當 message queue 用，也可以當 stream 用。

兩者的關鍵差別在消費模式。當 message queue 用時，每則訊息由 group 裡的一個 consumer 處理後就算「被消費」（雖然 Kafka 還是會依 retention 保留）；當 stream 用時，consumer 持續即時處理新進訊息，同一份資料可以被多個獨立的 consumer group 讀取，或從 log 任一時點重播。

以下情況可以考慮在設計裡加入 message queue：

- 有可以非同步處理的工作。例如 YouTube：使用者上傳影片後我們可以先提供標準畫質，再把影片（透過連結）丟到 Kafka topic，等系統有空再轉檔。
- 需要保證訊息按順序處理。例如 Design Ticketmaster 的虛擬排隊，可以用 Kafka 讓使用者依到達順序進入預訂頁。
- 想讓 producer 和 consumer 解耦、各自擴展。常見於 microservices：避免單一服務拖垮另一個。

Stream 適合這些情境：

- 需要對進來的資料做連續、即時的處理。例如 Design an Ad Click Aggregator 裡即時彙總點擊資料。
- 同一批訊息要給多個 consumer 同時處理。例如 Design FB Live Comments 可以用 Kafka 做 pub/sub，把留言送給多個 consumer。

面試該懂的 Kafka 重點

Kafka 要學的很多，這裡只聚焦面試最常相關的部分。

這份講義內容不少，尤其對面試所需而言。不用一次全吞：如果你是 junior 或 mid-level，下面很多可能用不到；senior 最好熟悉接下來幾段；staff 以上若能掌握多數會加分，但這些都不是「過關必要」。

Scalability（可擴展性）

buildmoat.org

先理解單一 Kafka broker 的極限。面試時需要先估算吞吐量和訊息量，才能判斷要不要談 scaling。

首先，單則訊息的大小可以透過 `message.max.bytes` 設定，沒有硬上限，但建議保持在 1MB 以下，以利效能（減少記憶體壓力、較佳網路利用）。

在 system design 面試裡，把「大塊資料」塞進 Kafka 是常見 anti-pattern。Kafka 不是資料庫，也不是拿來存大檔的，而是存小訊息、供快速處理。例如設計 YouTube 時，上傳後要做 chunk 和轉檔；直覺可能想把影片放進 Kafka 讓 worker 非同步拉下來處理，這樣不好。比較好的做法是：把影片存到 S3 這類分散式檔案系統，在 Kafka 裡只放一則「指向 S3 位置」的小訊息，讓 worker 去處理。

在還不錯的硬體上，單一 broker 大約可存 1TB 資料、每秒處理約 100 萬則訊息（依訊息大小和硬體會變，僅作數量級參考）。如果你的設計不需要超過這個量，可能不必討論 scaling。

若要擴展，有幾種策略：

1. **Horizontal Scaling With More Brokers（加更多 broker 做水平擴展）**：最直接的方式就是加 broker。加 broker 時要確保 topic 有足夠的 partition 才能利用新機器；如果 partition 不足，就無法發揮新 broker 的並行能力。
2. **Partitioning Strategy**：這應該是面試裡 scaling 的重點，也是你在處理 Kafka cluster 時的主要決策（因為很多 scaling 在 managed 服務裡是動態處理的）。你要決定怎麼把資料分到各 broker，也就是選 message 的 key。Partition 的公式是：`partition = hash(key) % num_partitions`（預設用 murmur2）。選錯 key 會造成 hot partition、流量集中；好的 key 會讓資料較均勻分布在 partition 上。

實務上很多 scaling 細節可以交給 managed 服務（例如 Confluent Cloud、AWS MSK）處理，但底層概念還是要懂。

討論 scaling 時通常是以 topic 為單位，因為不同 topic 需求不同。有的 topic 高吞吐、需要多 partition 分散到多台 broker，有的低吞吐、單一 broker 就夠。要擴展某個 topic，就增加其 partition 數量。

How can we handle hot partitions?（Hot partition 怎麼處理？）

面試官很愛問。例如 Ad Click Aggregator：Kafka 存使用者點廣告的點擊事件，很自然會用 ad id 當 partition key。但當 Nike 推出 LeBron James 新廣告時，那個 partition 會爆量，形成 hot partition。

有幾種策略：

1. **No key（default partitioning）**：不提供 key 的話，Kafka 會用預設 partitioner 把訊息分散到各 partition（現代 client 常用 sticky 策略：先集中到一 partition 再

bytebytego.org

輪替，長期下來大致均勻）。缺點是無法保證相關訊息的順序。如果順序對你的設計不重要，這是好選項。

2. **Random salting**：在 ad id 上加隨機數或時間戳當 partition key，可以讓負載更均勻分散到多個 partition，但 consumer 端的彙總邏輯會變複雜。這常被稱為對 key 做「salting」。
3. **Use a compound key**：不用單一 ad id，改用 ad id 加上另一個屬性（例如地區或 user id 區段）組合成 compound key，可以讓流量更均勻，尤其在你找得到與 ad id 獨立變化的屬性時很有用。
4. **Back pressure**：依需求而定，一個簡單做法是讓 producer 減速。如果用 managed Kafka 服務，可能已有內建機制；如果是自建 cluster，可以讓 producer 檢查 partition 的 lag，過高時就減速。

Fault Tolerance and Durability（容錯與持久化）

選 Kafka 的理由之一可能是它的持久化保證。Kafka 靠 replication：每個 partition 會複製到多台 broker，一台是 leader、其餘是 follower。Producer 送出的訊息先寫入 leader，再複製到 follower；即使一台 broker 掛掉，資料仍在。Producer 的 **acks** 設定很重要：`acks=all` 表示要等所有 **in-sync replicas (ISR)** 都收到才回覆，持久化保證最強。

依需求可以設定 topic 的 replication factor（每個 partition 有幾份 replica）。常見是 3（1 leader + 2 follower），這樣掛一台 broker 還有兩份資料，並可以把某個 follower 提升為新 leader。

But what happens when a consumer goes down?（Consumer 掛掉會怎樣？）

Kafka 常被說「always available」。你也常聽到「Kafka is always available, sometimes consistent.」意思是「Kafka 掛掉怎麼辦」這種問題不太實際，面試官這樣問時你甚至可以禮貌地反問一下。

更實際的是 consumer 掛掉。Consumer 故障時，Kafka 的容錯機制可以這樣延續：

1. **Offset Management**：Partition 就是 append-only log，每則訊息有一個唯一的 offset。Consumer 處理完訊息後會把 offset commit 回 Kafka，等於在說「我處理完這則了」。Consumer 重啟時會從 Kafka 讀取上次 commit 的 offset，從那裡繼續，所以不會漏訊息（但若在 commit 前掛掉，可能 at-least-once 重複處理）。
2. **Rebalancing**：在 consumer group 裡，如果某個 consumer 掛掉，Kafka 會把 partition 重新分配給剩下的 consumer，讓所有 partition 都有人處理。

buildnoat.org

面試裡常要取捨的是「何時 commit offset」。例如 Design a Web Crawler：要等 raw HTML 確定存進 blob storage 再 commit，否則可能重做。Consumer 單次要做的事越多，失敗時重做成本越高，所以把 consumer 的工作壓到最小是好的策略——Web Crawler 那題我們就把 crawler 拆成兩階段：先下載 HTML，再解析。

Handling Retries and Errors（重試與錯誤處理）

Kafka 本身負責大部分可靠性（如上所述），但「把訊息送進、拿出 Kafka」的過程仍可能失敗，需要妥善處理。

Producer Retries（Producer 重試）

首先，我們可能在一開始就沒辦法把訊息送進 Kafka。錯誤可能來自網路、broker 不可用或暫時故障。Kafka producer 支援自動重試，可以這樣設定：

```
const producer = kafka.producer({
  retry: {
    retries: 5,            // 最多重試 5 次
    initialRetryTime: 100, // 重試間隔 100ms
  },
  idempotent: true,
});
```

建議同時開啟 idempotent producer，避免重試時重複送出同一則訊息。

Consumer Retries（Consumer 重試）

Consumer 端可能因各種原因無法處理某則訊息。Kafka 本身沒有內建 consumer 重試（但 AWS SQS 有），所以要自己實作。常見做法是：設一個專門的 topic 放失敗的訊息，由另一個 consumer 負責重試；若重試太多次再移到 dead letter queue (DLQ)。DLQ 就是用來存失敗訊息、之後再排查的地方。

buildmoat.org

Main Topic: Consumer reads and processes message.

Retry Topic: Move failed message have and any.

DLQ Topic: Dead Letter Queue - give up, store for later investigation

Performance Optimizations（效能優化）

尤其把 Kafka 當 event stream 用時，要留意效能，才能盡快處理訊息。

可以先做 batching：在單次 `send()` 裡送多則訊息。Kafka producer 本來就會在送出去前先 batch，減少網路開銷。也可以用 `sendBatch()` 一次送多個 topic 的訊息：

```
await producer.send({
  topic: 'my_topic',
  messages: [
    { key: 'key1', value: 'message1' },
    { key: 'key2', value: 'message2' },
    { key: 'key3', value: 'message3' },
  ],
});
```

另一種常見做法是壓縮訊息，在送訊息時設定 `compression`。Kafka 支援 GZIP、Snappy、LZ4 等。壓縮就是讓訊息變小、送得更快：

```
const { CompressionTypes } = require('kafkajs')

await producer.send({
  topic: 'my_topic',
  compression: CompressionTypes.GZIP,
  messages: [
    { key: 'key1', value: 'Hello, Kafka!' },
```

buildmoat.org

```
  ],
});
```

對效能影響最大的往往還是 partition key 的選擇：讓訊息盡量均勻分布在 partition 上，才能最大化並行度。面試時討論 scaling，從 partition strategy 談起就對了。

Retention Policies（保留策略）

Kafka topic 有 retention policy，決定訊息在 log 裡保留多久，由 `retention.ms` 和 `retention.bytes` 設定，預設是 7 天。

面試可能被要求設計需要更長保留時間的系統，這時可以把 retention 調長，但要考慮對儲存成本和效能的影響。

總結

Apache Kafka 是開源、分散式的 event streaming 平台，強調高效能、可擴展和持久化。Producer 把訊息送到 topic，consumer 從 topic 讀取；訊息存在多台 broker 上、有序且不可變的 partition 裡，很適合 system design 中的即時資料處理和非同步 message queue。

談 scale 時，記得先從 partitioning strategy 和 hot partition 的處理方式講起。還有，Kafka 優先保證可用性而非一致性（CAP 定理中的 AP 系統）。

buildmoat.org